Research

Table of Contents

Abstract 3
Introduction 3
Research Question 4
Strategy 4
A. Choosing a Research Method 4
Introduction 4
1) The Library Method 4
2) The Field Method 4
3) The Lab Method 5
4) Showroom Method 5
Conclusion 5
B. Sources of information 6
C. Experts' Help 6
Research & Result 6
A. Determining Main Source of Information 7
B. Filtering Out Unrelated and Complex Content 7
Conclusion 8
C. Defining Key Research Questions 8
a) An introduction to Device Drivers 8
Chapter goal 9
b) Building and Running Modules 9
Chapter goal 9
c) Char drivers 9
Chapter goal 9
Conclusion 9
D. Research Results 10
1) An introduction to device drivers 10
a) What is the role of the driver? 10

b) What is the difference between mechanism and interface? 10
c) What are the main classes of Linux Devices and Modules? 10
d) What general security concepts should be followed while developing a driver? 11
2) Bulding and Running Modules 12
a) How to build a module? 12
b) What are the differences between a kernel module and an application? 12
c) What is the difference between User Space and Kernel Space? 13
d) How is a kernel object compiled and linked to the Kernel? 13
e) What is the kernel symbol table? 14
f) What does the initialization and shutdown process involve? 14
g) How can module parameters be set up at load time? 14
h) What are the advantages and limitations of a user space driver? 15
3) Char Drivers 16
a) What is SCULL? 16
b) What are the major and minor numbers used for? 16
c) What data structures are important for char drivers? 17
d) What does the registration process of a character device driver involve? 18
e) What SCULL methods have to be implemented to make the software driver work as expected? 20
f) What is the memory usage of SCULL? 23
a) How can SCULL be tested? 23
Research Conclusion 24
References 24

Table of Figures

Figure 1: Data Flow in SCULL 20
Figure 2: Example of container_of 22
Figure 3: SCULL Memory Buffer Organization 23

Abstract

The goal of the research is to understand the principles and design philosophy behind the development of a Linux device driver. As a result, the process of making the PCF8574 device controllable from a user-space application must be cleared up. The method that will be used is the Library one, because of the complexity of the topic. The main source of information will be Linux Device Drivers, 3rd edition by Corbet, Rubini and Kroah-Hartman. When information from there is not sufficient, expert teachers will be asked, and Linux Kernel Documentation will be read. The research found that the distinction between a policy and a mechanism is subtle for a good driver design. The goal of a driver is to enforce as few policies as possible. The driver must be programmed carefully, because bugs can result in system compromise, instability or hardware brick. The device drivers in Linux are split into 3 different classes - Character Devices, Block Devices and Network Interfaces. A driver is typically implemented as a kernel module, but it can also be developed as a normal application, depending on its goals - the first option is considered to be the better one in general, but exceptions exist. Kernel Modules differ significantly from normal applications. Their build process is complex; the kernel enforces its own special rules and involves additional steps. Programming in kernel space has its limitations and is different compared to user space application development. Proper error handling during module initialization is important. If it’s not performed correctly, the system can become unstable. A character driver provides a bridge of communication between user-space applications and the kernel. It provides methods which the kernel could call when a request is received by a user-space process. The research concluded that an efficient and reliable way of controlling the PCF8574 device driver is by exposing it to normal applications as a character device. That way, normal applications can use standard file operations (read/write/ioctl) and a custom communication protocol to control its behavior.

Introduction

The goal of the research is to understand how to design and develop a device driver in a systematic, modularized and optimized way, and a user-friendly interface that will provide easy and reliable control of the GPIO Expander device PCF8574. This research will build the foundation required to dive deeper into High-Performance Critical Systems.

Research Question

What are the design principles of a Linux driver and how can a PCF8574 device driver for a Linux-based Operating System be controlled from a normal application?

Strategy

A. Choosing a Research Method

Introduction

The DOT Framework provides many different possibilities to conduct research. The method that fits the best will be determined by a Positive or Negative answer of a question and a following explanation why.

1) Is the method applicable for the deep technical research that is required by the project?

1) The Library Method

Q: Is the library method applicable for the deep technical research that is required by the project?

A: Yes, it fits the needs of the project, because the complex concepts that the project requires cannot be grasped by trying out and understanding how everything works along the way. It requires knowledge in a very niche area of IT that not a lot of people are interested in or understand.

2) The Field Method

Q: Is the field method applicable for the deep technical research that is required by the project?

A: No, the field method wouldn't provide a lot of benefits for the project, mainly because of its deep dive into analyzing statistics, reaching out to stakeholders and doing surveys. This is not applicable to the problems that the development of the driver will face.

3) The Lab Method

Q: Is the lab method applicable for the deep technical research that is required by the project?

A: Partly yes, but it doesn't fit well enough to be the main method of research. It is oriented into finding the most useful way of the validation phase of the project.

4) Showroom Method

Q: Is the showroom method applicable for the deep technical research that is required by the project?

A: No, it is not applicable to the project, because its aim is to learn as much as possible from it, comparing it to others is outside of the project's scope.

5) Workshop Method

Q: Is the workshop method applicable for the deep technical research that is required by the project?

A: No, because it doesn't fit the goal of the project. It is not about solving a problem which people face, it is solved already. Its goal is to build foundational knowledge in a very niche area in IT.

Conclusion

The Library Method fits the research goals of the project better than all the other ones. The research will be conducted by a combination of studying literature and communicating with experts, because this combination is very effective when topics are both complicated and part of a niche area.

B. Sources of information

1) **Linux Device Drivers, 3rd Edition**
    - Although published in 2005 and focused on kernel v2.6, this book remains one of the best starting points. Despite 20 years of kernel evolution, the fundamental concepts remain the same. One of its co-authors is Greg Kroah-Hartman, maintainer of the stable kernel releases and various subsystems, which makes this source especially reliable.

2) **Kernel Source Code**
    - Available on GitHub, the Linux kernel source code is an invaluable reference. Learning how to navigate and understand a large codebase is essential for professional development.

3) **Kernel Documentation**
    - Since *Linux Device Drivers, 3rd Edition,* covers an older kernel, some aspects may differ in modern versions. The official documentation of the kernel running on my system will serve as the most accurate and up-to-date reference.

C. Experts' Help

1) Jaap Geurts
    - Mr. Geurts will be reached out whenever literature is not sufficient to understand a topic. His experience and knowledge align with the goals of the project.

Research & Result

A. Determining Main Source of Information

The main research resource will be LDD3. It is over 600 pages in total and covers not only basics, but advanced concepts as well. Chapters after 11 will be discarded from the research completely, because of their complexity they cannot be covered effectively throughout the duration of the project. To determine which chapters of the book must be covered, 2 Yes/No questions will have to be answered:

1) Is the chapter covering basics?
2) Is the chapter relevant to the project?

If both questions are answered positively, the material will be studied deeply.

Skimming throughout the pages is found to be ineffective when the reader doesn't have prior knowledge of the topic. Taking notes will be mandatory - the speed of studying the material will be slower, but more effective. Special attention will be paid to examples and figures, because they show the theory in practice and learning this is crucial for the outcome of the project.

The research won't give a direct solution to the problem but will find the standardized process of designing and implementing a driver. That knowledge can be applied to the same class of drivers that provide different facilities.

B. Filtering Out Unrelated and Complex Content

Chapter	Is the chapter covering basics?	Is the chapter relevant to the project?	Will the Chapter be studied?
1. An Introduction to Device Drivers	Yes	Yes	
2. Building and	Yes	Yes	

Running Modules			
3. Char drivers	Yes	Yes	
4. Debugging Techniques	Yes	No	
5. Concurrency and Race Conditions	No	No	
6. Advanced Char Driver Operations	No	No	
7. Time, Delays, and Deferred Work	No	No	
8. Allocating Memory	No	Yes	
9. Communicating with Hardware	No	Yes	
10. Interrupt Handling	No	Yes	
11. Data Types in the Kernel	No	Yes	

Conclusion

Out of the 11 possible chapters, only 3 are found to match the goal of the project and the expected level of complexity.

C. Defining Key Research Questions

a) An introduction to Device Drivers

a) What is the role of a driver?
b) What is the difference between Mechanism and Interface?
c) What are the main classes of Devices and Modules?
d) What general security concepts should be followed while developing a driver?

Chapter goal

The chapter will build the foundational knowledge about Device Drivers.

b) Building and Running Modules

a) How to build a module?
b) What are the differences between Kernel Modules and Applications?
c) What is the difference between Kernel Space and User Space?
d) How is a kernel object compiled and linked to the Kernel?
e) What is the Kernel Symbol Table?
f) How is a module initialized and removed?
g) How are modules setting up parameters at load time?
h) What is a user-space driver?

Chapter goal

The chapter will cover important module concepts that are crucial for every device driver, regardless of its class or facilities it provides.

c) Char drivers

a) What is SCULL?
b) What are the major and minor numbers used for?
c) What data structures are important for char drivers?
d) What does the registration process of a character driver involve?
e) How are SCULL methods implemented?
f) What is the memory usage of SCULL?
g) How can SCULL be tested?

Chapter goal

This chapter will show the development process of a simple character driver called SCULL (Simple Character Utility for Loading Localities). It won't provide a mechanism for controlling a real device, but it can work as a template to follow while developing the PCF8574 Device Driver.

Conclusion

The research questions are a lot and all of them are found to be crucial to the design and development process of the Device Driver. Answering them will provide the reader not only with

key knowledge from the book, but also with reasoning behind every decision throughout designing and developing the device driver.

D. Research Results

1) An introduction to device drivers

a) What is the role of the driver?

Answer

- The role of a driver is to provide an internal programming interface used by the operating system to control hardware or software devices. (LDD3, p.1)
- Software devices provide additional capabilities without having to rely on additional hardware. (LDD3, p.1)

Example

The loopback interface in Linux is an example of a software device driver. If a user sends packets to 127.0.0.1 (localhost), the packets are handled by the kernel as if they come from outside the Machine, but in reality, they come from the same one. (LDD3, p.1)

b) What is the difference between mechanism and interface?

Answer

- In the design of UNIX, the distinction between mechanism and interface is foundational. The mechanism provides the capabilities, and the interface provides policies on how to use them. That approach makes it possible for different applications to use the provided driver in unique ways. (LDD3, p. 2)
- The device driver must avoid applying policies as much as possible. The more freedom an interface developer has, the better the design of the driver is. (LDD3, p.2)

c) What are the main classes of Linux Devices and Modules?

Answer

- According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 6-7), the main classes of Linux devices and modules are divided into three categories:
  1) Character Devices
  2) Block Devices
  3) Network Interfaces

1) *Character Device*
    - A device that can be accessed as a stream of bytes in the /dev/ directory. It implements the system calls open(), write(), read(), close() and others. A subtle difference between a character device and actual files is that inside a file you can move back and forth, while character devices mostly are just data channels, which can be accessed only sequentially. (Corbet et al., 2005, LDD, p. 6)
2) *Block Device*
    - Can be accessed in the same way as a character device. It is a device which can host a filesystem. In most UNIX Systems, a block device can only handle I/O Operations that transfer 1 block minimum, (size of a block is typically 512 bytes). In Linux, they can transfer any number of bytes. For user space applications, they look the same as character devices, but their implementations in the kernel are completely different from one another. (Corbet et al., 2005, LDD, p. 6)
3) *Network Interface*
    - It is a device which is in charge of sending and receiving data packets. It doesn't know how those packets map to the processes that use them, it just provides the infrastructure required for the kernel to handle them. They cannot be accessed as a filesystem node, and they completely differ from character and block devices. (Corbet et al., 2005, LDD, p. 6)

d) What general security concepts should be followed while developing a driver?

- According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 8-9), while developing device drivers, security holes can result in System Compromise or Hardware Damage.

To avoid those issues, the authors suggest following 6 principles

1) Avoid implementing security policies in drivers when possible, as security should be enforced at higher kernel levels.
2) Provide adequate control for device access that can affect system behavior, to prevent misuse.
3) Drivers must be carefully programmed to avoid memory bugs, as C does not prevent such errors. Such bugs can destabilize the system and may be exploited to compromise it.
4) All user input must be treated as untrusted until validated.
5) Avoid using uninitialized memory; memory obtained from the kernel must always be zeroed to prevent information leakage.
6) Restrict device operations that can affect system integrity to privileged users only.

2) Bulding and Running Modules

a) How to build a module?

Answer

- Building a module requires having the kernel source tree on the device's system. (Corbet et al., 2005, LDD, p. 15)
- The module is linked against object files found in the kernel source tree. This results in a more robust module loader compared to earlier than 2.6 versions. (Corbet et al., 2005, LDD, p. 15)
- Unless a driver is developed for a specific distribution, it is recommended to use the mainline kernel, as vendors often change the API of the Kernel to suit their needs. (Corbet et al., 2005, LDD, p. 15)
- It's highly recommended to run the experiment modules on a system whose files are affordable to be lost or corrupted, because modules could potentially create serious problems, depending on the bug they create. (Corbet et al., 2005, LDD, p. 16)

b) What are the differences between a kernel module and an application?

Answer

According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 18-19), the most crucial differences are 5:

a) Concurrency

- While small to medium sized applications typically use only one thread to execute, even small drivers must be designed with Concurrency in mind, because there is never a guarantee that the driver will be used only by one application at a time.

b) Interrupts

- User-space programs don't have to worry about handling interrupts, because the Linux Kernel does that for them. It is a preemptive kernel, which basically means that every process in the system can be interrupted at any given time when a higher-priority process must be executed. Drivers must handle interrupts manually.

c) Stack size

- Kernel-space programming differs significantly from User-Space. The kernel and its modules operate in a very limited stack - around the size of a page (4096 bytes). This removes the possibility to declare large amounts of variables on the stack. Because of that, most of the variables must be allocated on the heap. In most cases, kernel-space programs must only use the functions which the kernel provides, while user-space applications have a wide choice of hundreds or thousands of libraries.

d) Build Process

- The build process of a user-space C application differs a lot from the one in the Kernel-Space. While going into details will be hard, it's important to point out why the build process differs. The kernel allows dynamic loading and unloading of Kernel Modules. That means that there is one extra step than the usual process (the object (.o) file has to be turned into a kernel object (.ko) file). The .ko file contains important information required by the kernel, so the module can be incorporated and used properly.

e) Debugging

- Bugs in the kernel can be fatal and debugging them is a lot harder than those of user-space software. A simple segmentation fault in a user-space application just shuts down the program, while a segmentation fault in the kernel might crash the whole system.

c) What is the difference between User Space and Kernel Space?

Answer

According to According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 19-20):

- A subtle difference between them is that normal applications cannot use all available CPU instructions. Some of them are reserved only for the Operating System. Furthermore, they have their own memory space. Applications can be executed on behalf of the kernel, whenever a system call is executed.

d) How is a kernel object compiled and linked to the Kernel?

Answer

According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 23-24), the building process of a module differs significantly from the one of a user-space application.

- The building process of a module differs significantly from the one of a user-space application. One of the crucial reasons for that is because a module is being linked to the running kernel, which is a standalone program that enforces explicit requirements on how its pieces are put together.
- The module must be compiled by the same version of the compiler that was used for the kernel.
- The syntax of make is different as well. The main reason for that is because a module must use the kernel source tree to be built.
- In reality, the build process is highly complex and the role of the makefile is to invoke the kernel source tree, which is doing the actual linking and compilation.
- Object file (.o) is not enough, it has one more compilation step, which turns the .o file into .ko (kernel object). After the success of that operation, the module can be inserted in the kernel at runtime.

e) *What is the kernel symbol table?*

Answer

According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 28-29), the kernel symbol table contains addresses of global functions and variables.

- Even if a module function is not static, it still cannot be called from outside the module. To make functions and variables accessible by other parts of the kernel, the macro EXPORT_SYMBOL(name) must be used.

f) *What does the initialization and shutdown process involve?*

Answer

According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 31-34), initialization and shutdown are of vital importance for the correct behavior of a module. The module_init(init_function) and module_exit(exit_function) macros are defined to tell the kernel which module function to call when a module is linked and removed from it.

a) Initialization

- The initialization function can be marked as __init. This will tell the kernel that this function is only for initialization, and after it's executed, the memory that it holds is freed.
- The goal of the initialization function is to prepare the kernel module for use by the kernel. Each module must be registered a type of facility that it provides. That includes different devices, filesystems, cryptographic transforms and more.
- Many of those registerable items aren't related to hardware, but provide software abstractions. Example for that are the /proc files, used to obtain information from the kernel.

b) Shutdown

- The goal of the shutdown function is to free the resources, which were obtained by the module while it was running in the kernel.
- The shutdown function could be marked as __exit. That will tell the kernel that the function is used only to remove the module from the kernel. It's important to remember that this function shouldn't be marked as such, if it is called from somewhere else. An example for that is when something went wrong during initialization. It's highly recommended to keep the module in the kernel with degraded functionality, but if it is not a viable option, then everything initialized so far must be cleaned up.

g) *How can module parameters be set up at load time?*

Answer

According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 35-37), setting up parameters at load time is important, because it provides flexibility on how the driver is used.

- Parameters can be loaded in the kernel at load time by setting up the desired parameter with the module_param() macro. It has 3 fields, name of the variable, type of the variable and sysfs permissions. Then, during loading, additional parameters must be passed to the insmod command in the terminal. Example: insmod module.ko name="Georgi". This will modify the default content of the variable name to "Georgi".

h) What are the advantages and limitations of a user space driver?

Answer

According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 37-39), a user space driver has both advantages and disadvantages, and developing such is a reliable option in some cases.

a) Advantages

- Writing a closed-source driver in user space avoids ambiguous licensing situations.
- A user space driver is not dependent directly on kernel interfaces and when they change, the driver will still work.
- A user space driver can use the standard C library, which has a lot of features that can speed up its development.
- A user space driver is far easier to debug and bugs cannot harm the system's stability.

b) Limitations

- Response time is slower compared to a kernel space driver, because context switching introduces overhead.
- If the user driver is moved to the disk, which will happen if the driver is not used often, the response time will be very slow. The reason for this is because the data must be transferred from the disk to the RAM.
- If the process is not run by a privileged user, it cannot access I/O ports or have direct access to a chunk of memory.
- Interrupts are not available in user space. There are workarounds by using the vm86 call.
- Most important devices such as network interfaces and block devices cannot be handled from user space.

3) Char Drivers

a) What is SCULL?

Answer

- According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 42-43), Simple Character Utility for Loading Localities (SCULL) is a software device driver designed by the authors of the book to showcase the basics of a driver design and implementation. It has 6 variations:
    a) scull0 to scull3
    b) scullpipe0 to scullpipe3
    c) scullsingle
    d) scullpriv
    e) sculluid
    f) scullwild

    a) scull0 to scull3
        - Four software devices that provide global and persistent memory area, which can be used by processes to read and write information on.

    b) scullpipe0 to scullpipe3
        - Four FIFO (First In, First Out) devices, which act like pipes. One process reads what another process writes. If multiple processes want to use the device, a race condition will occur. It will explain how block I/O could be implemented without using interrupts.

    c) scullsingle
        - Same as a) but only one process can open it at a time

    d) scullpriv
        - Same as a) but it is private to each virtual console.

    e) sculluid
        - Same as a), it can be opened by multiple processes, but only by 1 user. If the device is used by another user, accessing it results in “Device Busy”

    f) scullwild
        - Same as a), it can be opened by multiple processes, but only by 1 user. If the device is used by another user, accessing it results in a blocked open.

b) What are the major and minor numbers used for?

Answer

- According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 43-44), major and minor numbers are of crucial importance for char and block device drivers. The combination of those numbers helps the kernel find the file operations of the driver and the device that uses them.

a) *Major number*
   - It represents the id of a driver.

b) *Minor number*
   - A driver could be used by more than 1 device, so one major number can have many minor numbers.

c) *What data structures are important for char drivers?*

Answer

- According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 49-55), the important data structures for char drivers are 3:
  a) struct file_operations
  b) struct file
  c) struct inode

a) *struct file_operations*
   - Defined in <linux/fs.h>, the structure provides the bridge between a process and the functionalities of the driver. It consists of function pointers that must point to the driver's functions.

b) *struct file*
   - Defined in <linux/fs.h>, the structure is initialized by the VFS (Virtual File System) layer when an open() system call is executed by a user-space process. It can be viewed as an object instance, because the file_operations struct provides the methods that operate on the data of the file struct. Every struct file is unique to each open file descriptor.

     For example, if Process A writes to the device driver, and Process B reads from it, they need to have separate file structs, because if Process A wrote 30 bytes, Process B would start reading from byte 31, which wouldn't make any sense and will result in Process B not reading any information.

c) *struct inode*
   - Defined in <linux/fs.h>, the structure is used to represent files internally. Unlike the struct file, It's not representing a file descriptor. The inode struct includes a pointer to the registered character device driver, so all file descriptors that use the same device driver, include a pointer to the same inode struct. That's foundational for allowing more than one process to use the device driver.

d) What does the registration process of a character device driver involve?

Answer

- According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 55-56), the registration process of a character device driver involves 3 steps:
    a) Allocating/Choosing a major number
    b) Initializing struct cdev
    c) Registering struct cdev as usable by the kernel

a) Allocating a major number

- A major number can be allocated dynamically and manually, but it is considered best practice to always let the kernel decide what major number must be used by the device driver. Choosing a major number manually can be tedious, because one major number, which is free on one device, is used by something else on another.

1) Dynamic Major Number Allocation

- To allocate a major number dynamically, the function **int alloc_chrdev_region(dev_t dev*, unsigned int firstminor, unsigned int count, char *name)** is used. The first argument **dev_t dev*** exists so the developer knows what is the major and minor number of the first device driver. **unsigned int firstminor** and **unsigned int count** are responsible to register more than one device driver correctly, if requested. The last argument **char *name** is the name of the device that is associated with the number range. It appears in /proc/devices and sysfs.

2) Manual Major Number Allocation

- To allocate a major number manually, **int register_chrdev_region(dev_t first, unsigned int count, char *name)** is used. The major difference compared to a) is that the **dev_t** variable (which includes both major and minor numbers) must be assembled by using the macro **MKDEV(major_number, minor_number)**, prior to using the function.

b) Initializing the cdev struct

- This step is mandatory before telling the kernel that it can use the driver. The function used to initialize the cdev struct is **cdev_init(struct cdev *dev, struct file_operations *fops)**.
  The first argument is the target cdev struct and the second one is a pointer to the file operations of the device driver, so the character device is associated with the correct file operations.
  This feature allows the same type of device drivers to have different file operations. In OOP terms, this is called method overriding.

c) Registering struct cdev as usable by the kernel

- To achieve this, the kernel module has to use the **cdev_add(struct cdev *dev, dev_t num, unsigned int count)** function. The first argument is a pointer to the initialized cdev struct, the second one is the major and minor number that will be used by the kernel to find the correct cdev struct and the last one is the total number of devices that are going to be associated with it. Usually the last argument is almost always 1, but there are exceptions.
  Example:
  The SCSI tape driver allows user space to select operating modes by assigning minor numbers to each physical device.

e) What SCULL methods have to be implemented to make the software driver work as expected?

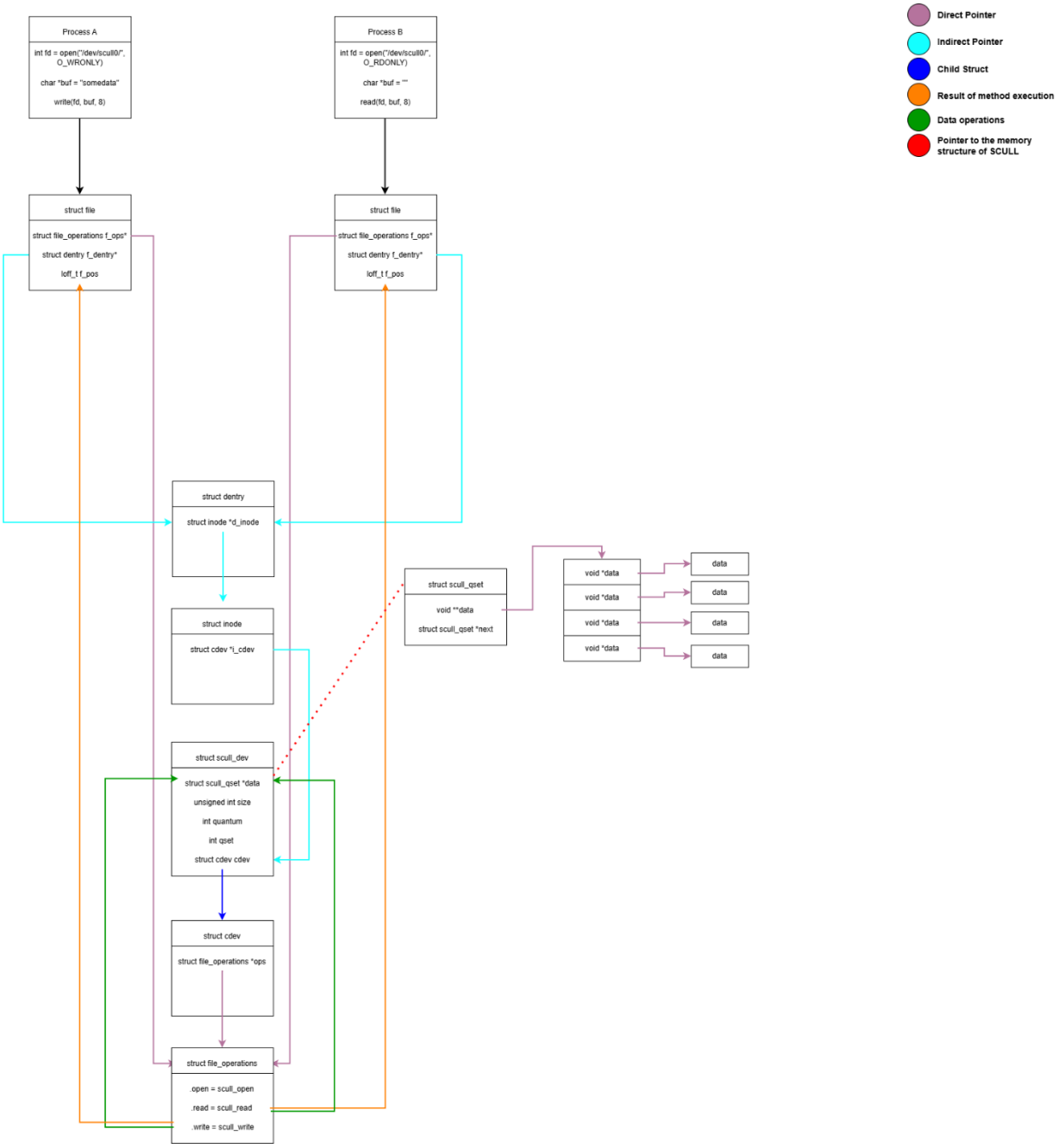


Figure 1: Data Flow in SCULL

Answer

- According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 58-60), the SCULL device driver has to provide an implementation of 4 methods:
    a) Open
    b) Release
    c) Read
    d) Write

*a) int scull_open(struct inode *inode, struct file *filp)*

- This method is the first one called by the kernel when a process wants to use the device driver. The implementation of this method has to prepare the file object for future use. Since the scull driver is designed to be used by 4 devices, the opened device has to be identified before it is used. To achieve that, the macro **container_of(pointer, container_type, container_field)** has to be used. It calculates the offset of the field from the struct, and then the result is the address of the first argument minus the calculated offset. Furthermore, the pointer to the struct of scull has to be saved in filp->private_data, because the inode pointer is not passed in every method.
  If the return value is a negative number, a problem has occurred during executing the method.

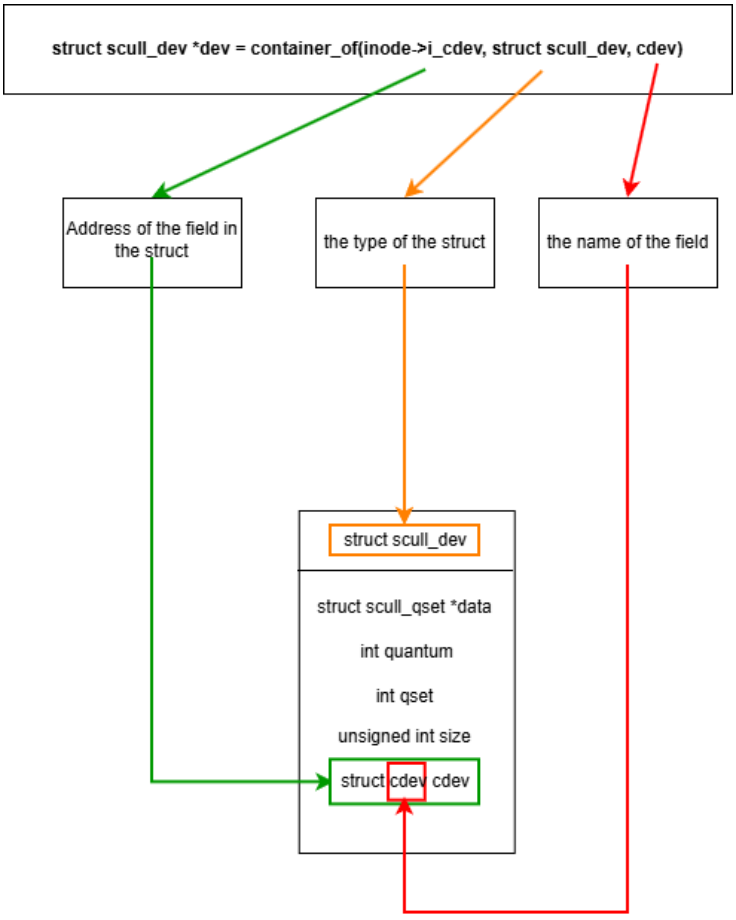


Figure 2: Example of container_of

b) *int scull_release(struct inode *inode, struct file *filp)*

- This method doesn't have to do a lot, because scull doesn't need to shutdown any hardware.
  If it returns a negative number, a problem has occurred during method execution.

c) *int scull_write(struct file *filp, const char __user *buf, size_t count, loff_t *f_pos)*

- This method has to write data incoming from user-space to the memory of the device. The role of the **f_pos** pointer is crucial - it tells where the data must be written. If there is no space, heap memory has to be allocated.
  Returns the number of bytes successfully written. If it's a negative value, a problem has occurred. If it's 0, no bytes were written.

d) *int scull_read(struct file *filp, char __user *buf, size_t count, loff_t *f_pos)*

- This method has to read data from the memory of the driver and send it to the user space process that requested it. If the same process wants to read data

continuously, it will start reading from the 1st unread byte, because f_pos always tracks the amount of read bytes.

Returns the number of bytes successfully read. If it's a negative value, a problem has occurred. If it's 0, no bytes were read.

f) What is the memory usage of SCULL?

Answer

- According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (pp. 60-62), the memory usage of SCULL consists of a linked list, which contains an array of void pointers to blocks of memory. The size of the array and the block is policy as the user is left with a decision on what their size should be. The driver provides a standard configuration - one struct scull_qset contains 1000 pointers that point to 4000 bytes each.

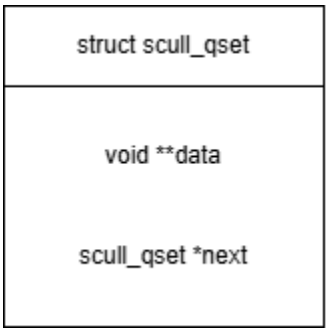


Figure 3: SCULL Memory Buffer Organization

a) How can SCULL be tested?

Answer

- According to *Linux Device Drivers, third edition* by Corbet, Rubini and Kroah-Hartman (p. 70), the software device driver SCULL can be tested in different ways.
    a) By using the command cat in the linux terminal.
    b) By developing a user-space program that reads or writes (or both).
    c) By using cp to copy data from a file to the device, or vice-versa.

Research Conclusion

The PCF8574 device driver should follow standard Linux device driver design principles. It has to provide a mechanism and must avoid enforcing policies on how it can be used. A PCF8574 device driver can be controlled from normal applications by exposing itself to user space as a character device. This would allow processes to interact with the driver via the system calls read, write and ioctl. The kernel will invoke the functionality of the driver, which the process has requested. The design of a clear communication protocol between the process and the driver is essential for the proper control of the PCF8574 device.

References

[1] **HBOi. (2021, July). The DOT Framework - ICT research methods**
https://ictresearchmethods.nl/dot-framework/

[2] ***Corbet, J., Rubini, A., & Kroah-Hartman, G. (2005). Linux Device Drivers (3rd ed.). O'Reilly Media.***
https://lwn.net/Kernel/LDD3/